החוג למערכות מידע
הפקולטה למדעי המחשב והמידע
אוניברסיטת חיפה

Software Engineering and Software Quality Assurance in Information Systems

Assignment 2

Software Requirements Specification (SRS)

For Ludo Educational Game

Panther Group

Member	Student ID	GitHub username
Islam Hassadya	206858334	IslamHassadya11
Fahid Abu Khadra	325558922	Fahidak22
Rami Aboud	214586208	Ramiaboud2005-dotcom
Ghazal Nahhas	212242937	ghazalnah

Repository Link: https://github.com/IslamHassadya11/SE-Ludo-Panther

Department of Information Systems

University of Haifa

2026-2027

Table of Contents

1. Introduction 1
1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms and Abbreviations 1
1.4 References 3
1.5 Overview 4
2. Overall Description 4
2.1 Product Perspective 4
2.2 Product Functions 5
2.3 User Characteristics 6
2.4 Constraints 6
2.5 Assumptions and Dependencies 6
2.6 Apportioning of Requirements 6
3. Specific Requirements 7
3.1 External Interface Requirements 7
3.2 System Features 9
3.3 Performance Requirements 16
3.4 Design Constraints 16
3.5 Software System Attributes 17
3.6 Other Requirements – Logical Database Requirements 18

1. Introduction

1.1 Purpose (Fahid)

This SRS document specifies the software requirements of Ludo, an educational computer version of the classic board game, developed by group Panther as the semester project of the course Software Engineering and Software Quality Assurance in Information Systems. It is the agreed statement between the customer and the development team of what the system shall do and under which constraints, and the baseline for testing and approving the finished system [4][1]. It is intended for the course staff, who act as the customer and approve the requirements, and for the development team, who design, build and test the system from it. Section 1.3 defines the game and technical terms needed to read this document correctly.

1.2 Scope (Fahid)

Ludo is a desktop game for 2–4 players sharing one computer, with Easy, Medium and Hard levels. Each level sets the board size, special squares and dice. Easy is classic Ludo with protection squares; Medium and Hard add question squares, which ask multiple-choice questions on the course topics (Agile, MVC, design patterns, quality assurance, software design), and surprise squares, which trigger random events. Every finished game is saved to a game history file that can be viewed later, and a password-protected question wizard lets the admin manage the question bank.

Out of scope for this version: network play, computer-controlled opponents, saving and resuming an unfinished game, player accounts and mobile versions.

Goal: to let students practice the course material in an enjoyable, competitive way.

1.3 Definitions, Acronyms and Abbreviations (Islam, Ghazal, Rami)

Term	Definition
Admin	The person authorized to manage the question bank.
AES-GCM	A standard encryption method that also detects any change to the encrypted data.
Arrow square	A square that turns pieces of its color onto the next inner track or into the home lane.
Boost die	A die with values 1–3, rolled with the regular step die on Medium.
Byte	A basic unit of computer data, made of 8 bits.
Castle	A player's corner area, where pieces start and return when eaten.
Chi-square test	A statistical test of whether results occur as often as expected.

[1] Numbers in square brackets, such as [4], refer to the references in section 1.4.

Class	A Java program unit grouping related data with its actions.
Contrast ratio	The brightness difference between text and its background.
Controller	In MVC, the classes that receive the user's actions, ask the Model to apply them and update the View.
CSV	Comma-Separated Values: a text file storing a table, one record per line, with fields separated by commas.
Design pattern	A standard, reusable code structure for a recurring design problem.
Double Block	Two or more pieces of one player on a square, which opponents can neither reach nor pass.
Eating	Landing on a lone, unprotected opponent piece and sending it to its castle.
Encryption	Encoding data so that only someone holding the secret key can read it.
Finish	A player's home square at the board's center, where a piece's route ends.
Hash	A fixed-length, irreversible code calculated from a password.
Home lane	A player's six-square path to its home square.
Home square	The protected last square of a player's route, ending its home lane.
IEEE	Institute of Electrical and Electronics Engineers: a professional body that publishes engineering standards.
IEEE 830	The IEEE recommended practice for writing an SRS, followed here.
JAR file	Java Archive: a single runnable file that packages a Java program.
Java 21	Version 21 of Java, the programming language the game is written in.
Java Swing	The standard Java library for building screen elements.
Javadoc	Code comments from which Java generates documentation automatically.
JRE	Java Runtime Environment: the software required to run Java programs.
LTS	Long-Term Support: a Java version updated for several years.
macOS 13	Version 13 of Apple's desktop operating system.
Model	In MVC, the classes that hold the data and the game rules.
MVC	Model-View-Controller: an architecture that divides a program into three parts – data and rules, screens, and handling of user actions.
PBKDF2	A standard way to hash a password many times, making guessing slow.
Pixel	The smallest dot on a screen; the unit of screen size.

Protected square	A square where pieces cannot be eaten: protection, start, home or surprise.
Protection square	A star-marked protected square.
Question bank	The stored collection of all the game's questions.
Question square	A "?" square that shows a question: green for easy, yellow for medium, red for hard. Pieces on it can be eaten.
Question wizard	The admin's protected screens to view, add, edit and delete questions.
Salt	A random value added to each password before hashing, so that identical passwords produce different hashes.
Shared track	A path used by all players: one on Easy, two on Medium and three on Hard.
Special square	An arrow, home, protection, question, start or surprise square.
SRS	Software Requirements Specification: a document of what a system must do.
Start square	A #-marked protected square where pieces enter from the castle.
Step die	A regular die with values 1–6, used on every level.
Surprise square	A protected square that triggers a random event.
UltraBoost die	A multiplier die with values 1–6, rolled with the step die on Hard and in one surprise event; the step die result is multiplied by its result minus 1.
Unit test	A small automatic test that checks that one part of the code works correctly.
UTF-8	The standard way of storing text as bytes, supporting all languages.
View	In MVC, the classes that display the screens.
WCAG	Web Content Accessibility Guidelines: international guidelines for making screens usable for people with disabilities [10].
Windows 11	Version 11 of Microsoft's desktop operating system.

1.4 References (Rami, Islam, Fahid, Ghazal)

[1] Course staff. (2026a). *Ludo*. Department of Information Systems, University of Haifa.

[2] Course staff. (2026b). *History DataBase Template*. Department of Information Systems, University of Haifa.

[3] Course staff. (2026c). *Answers to students' questions about the project*. Department of Information Systems, University of Haifa.

[4] IEEE. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). https://doi.org/10.1109/IEEESTD.1998.88286

[5] Laubheimer, P. (2020). *Executing UX animations: Duration and motion characteristics*. Nielsen Norman Group. https://www.nngroup.com/articles/animation-duration/

[6] Microsoft. (n.d.). *Windows 11 specifications and system requirements*. https://www.microsoft.com/windows/windows-11-specifications

[7] National Institute of Standards and Technology [NIST]. (2025). *Digital identity guidelines: Authentication and authenticator management* (NIST SP 800-63B-4). https://doi.org/10.6028/NIST.SP.800-63B-4

[8] Nielsen, J. (1993). *Response times: The 3 important limits*. Nielsen Norman Group. https://www.nngroup.com/articles/response-times-3-important-limits/

[9] OWASP Foundation. (n.d.). *Authentication cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[10] W3C. (2023). *Web content accessibility guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

[11] Anthropic. (2026). *Claude* (Opus 5.5 version) [Large language model]. https://claude.ai[2]

1.5 Overview (Rami)

After this introduction, the document is organized as follows:

- **Section 2** describes the product as a whole.
- **Section 3** gives the detailed requirements, **organized by feature** [4, annex A.5]; the reasons for this choice open the section.

Requirement identifiers: FR-nn for functional and NFR-nn for non-functional requirements.

Importance of each functional requirement follows the degree of necessity defined in IEEE 830 [4, section 4.3.5.2].

- **3 – High (essential):** the software is not acceptable unless the requirement is provided as agreed.
- **2 – Medium (conditional):** the requirement enhances the software, but the software is acceptable without it.
- **1 – Low (optional):** the requirement adds value beyond the customer's request; the software is acceptable without it.

Every requirement comes from the customer's instructions [1]–[3] except team decisions marked by [TD].

2. Overall Description

2.1 Product Perspective (Ghazal)

The system is a new, self-contained product that runs as a single program on one computer.

[2] The team used Claude only for finding other relevant references and revising the grammar.
[TD] Team decision: a requirement the team decided on a point the customer left open.

2.1.1 System interfaces: not relevant, because the system exchanges no data with other systems.

2.1.2 User interfaces: the opening screen, the game setup screen, the game screen with its question and surprise windows, the game-over screen, the game history screen, the admin login and the question wizard (see section 3.1.1).

2.1.3 Hardware interfaces: only a standard screen, keyboard and mouse (see section 3.1.2).

2.1.4 Software interfaces: the JRE and the computer's file system (see section 3.1.3).

2.1.5 Communications interfaces: not relevant, because the game runs offline and keeps all its data on the local computer (see section 3.1.4).

2.1.6 Memory constraints: the game runs on a computer with 4 GB of memory, the reference computer of section 3.3 [6].

2.1.7 Operations: the system has two kinds of use, one for each user class: playing a game (players) and managing questions (admin). The customer requires no backup function; if the history file is missing or damaged, the system recovers as described in section 3.5.1.

2.1.8 Site adaptation requirements: not needed, because the same runnable file works on every computer.

2.2 Product Functions (Ghazal)

The system offers the following main functions to its two user classes (section 2.3); section 3.2 specifies each of them as a feature.

- **Configure and Start a Game** (players) – the users choose the number of players, the level and each player's name and color. The system then builds the level's board and places the protection, question and surprise squares at random.
- **Play a Ludo Game** (players) – the players take turns rolling the dice and moving their pieces, while the system applies the rule of 6, eating, protected squares, Double Blocks, the home lane and the exact roll needed to reach the finish. On Medium and Hard, question squares ask a question on the course material (a correct answer moves the piece forward, a wrong one back), and surprise squares apply one of six random events: jump to the finish, move an opponent's piece back, back to the castle, back 3 steps, lose the next turn, or an extra roll. When all players but one have finished, the system ranks the players, shows the result and saves the game.
- **View Game History** (players) – the system shows all finished games, which the user can sort and filter.
- **Manage the Question Bank** (admin) – the admin logs in to the protected question wizard to view, add, edit and delete questions and to change the password.

2.3 User Characteristics (Fahid)

Player: mainly software engineering and quality assurance students, but anyone who can use a mouse and keyboard and knows the course material can play. Since some players may not know Ludo's rules, the game explains them in a Rules window and allows only legal moves.

Admin: a person authorized to maintain the question bank, such as a course staff member or an appointed team member. The admin needs to know the course material and have basic computer skills; no programming knowledge is needed, since questions are managed in the question wizard. The admin may also take part in a game as a regular player, without admin privileges.

2.4 Constraints (Fahid, Islam)

The system is subject to the following constraints, detailed in sections 3.4 and 3.5.3:

- **Programming language:** Java 21 [1] (see NFR-18).
- **Architecture:** the code follows MVC, which keeps the game's data and rules, its screens and its handling of user actions in separate parts, and it uses proven design patterns that make the code easier to change and reuse, as the customer requires [1] (see NFR-20 and NFR-21).
- **Language:** English is the default screen language, with Hebrew and Arabic as optional languages (see FR-06).
- **Data format:** the game history keeps every field of the course staff's template (see section 3.6).
- **Security:** only the admin may access the question bank (section 3.5.3).
- **Delivery:** every version is delivered as a runnable JAR file, tested on several computers beforehand (see NFR-23).
- **Regulatory policies:** the customer sets no legal or regulatory requirements. The only personal data kept are the players' names in the history file, which never leaves the local computer, and the admin password is stored only as a salted hash (section 3.5.3).

2.5 Assumptions and Dependencies (Ghazal)

A1. The computer that runs the game has a screen of at least 1280×720 pixels; tablets and phones are not supported.

A2. The computer that runs the game has Java 21 or later installed.

A3. The program folder on that computer is writable, since the data files are kept there.

A4. Questions are shown in the language the admin wrote them in; the system does not translate them.

2.6 Apportioning of Requirements (Ghazal)

Requirements of importance 3 and 2 are planned for the final version, with importance 3 implemented first. Importance 1 requirements may be delayed to a future version if time runs short. The out-of-scope capabilities of section 1.2 are left for future versions.

3. Specific Requirements (Islam, Fahid, Rami, Ghazal)

Organization of this section. IEEE 830 offers eight templates for this section [4, annex A]; we chose organization by feature (annex A.5), where a feature is an externally desired service by the system that may require a sequence of inputs to effect the desired result, described by stimulus/response pairs [4, section 5.3.7.4]. Our system offers four such services: configuring and starting a game, playing it, viewing the game history and managing the question bank. Each feature in section 3.2 gives its purpose, its stimulus/response sequence and its functional requirements.

The other templates fit our system less well. Mode (A.1, A.2) would repeat every shared rule under each level, since the levels differ only in numbers such as board size, special squares and dice; A.2, which also gives each mode its own interfaces and performance requirements, would repeat even more. User class (A.3) would divide the requirements unevenly: the admin has one group of functions, and all other requirements belong to the players. Object (A.4) does not fit because many rules involve several objects: eating, for example, involves a piece, a square and a castle, so placing such a rule under one object would be arbitrary. Stimulus or response (A.6) does not fit because one stimulus, such as clicking Roll, leads to many different outcomes depending on the game situation, so each stimulus would need many sub-scenarios, and rules shared by several stimuli would be repeated. Functional hierarchy (A.7) is meant for data-processing systems, and IEEE 830 recommends it only when no other organization helps. Combining user class with feature (A.8) would add a whole level of headings above the four features, which our system does not need.

3.1 External Interface Requirements

3.1.1 User interfaces (Islam, Rami)

ID	Functional requirement	Imp.	Rationale
FR-01	On start-up, the system shall show the opening screen with New Game, Game History, Question Management, Settings and Exit.	3	Every other screen is reached from this one.
FR-02	New Game shall open game setup, Game History the history screen, and Question Management the admin login.	3	Without these links, the main screens could not be reached.
FR-03	Every screen except the opening one shall offer Back. Leaving a running game by Back or Exit shall require confirmation.	2	Prevents losing a game by mistake, but the game works without it. TD
FR-04	The game screen shall show the board, the game time (HH:MM:SS, updated every second), the current player's name and color, the dice results and a Roll button.	3	Players need this information to follow the game and take their turns.

FR-05	The game screen shall list the players in turn order, with each player's number of finished pieces.	2	Shows each player's progress; the game works without it. TD
FR-06	The system should offer its screens in English, Hebrew and Arabic, selected in the Settings screen. Hebrew and Arabic screens are shown right to left.	1	Extra languages are a convenience that the customer did not ask for. TD
FR-07	The game screen shall offer a Rules window that explains the squares, dice, questions and surprise events.	2	Helps players new to Ludo. TD

ID	Non-functional requirement
NFR-01	Every window shall be fully usable at 1280×720 pixels, the minimum display of current desktop computers [6], and above.
NFR-02	The board shall scale with the window, staying fully visible and readable.
NFR-03	Text shall be at least 12 pt, and the current player's name at least 16 pt.[3]
NFR-04	Text shall have a contrast ratio of at least 4.5:1 with its background [10, criterion 1.4.3].[4]
NFR-05	Color shall never be the only way to tell players apart; each player's color shall also be shown by its name [10, criterion 1.4.1].
NFR-06	Every button and selectable piece shall be at least 24×24 pixels [10, criterion 2.5.8].
NFR-07	Every pop-up error message shall state what is wrong and how to fix it.

3.1.2 Hardware interfaces (Islam)

Not relevant, since the system uses only a standard screen, keyboard and mouse.

3.1.3 Software interfaces (Islam)

ID	Functional requirement	Imp.	Rationale
FR-08	The system shall read and write the game history as a UTF-8 CSV file whose header row holds the template fields of section 3.6.	3	Without a fixed file format, saved games could not be read back.

ID	Non-functional requirement
NFR-08	The system shall run on Java 21 or later.

[3] 12 pt is a common size for readable screen text; 16 pt makes the current player easy to spot.
[4] A higher contrast ratio makes text easier to see.

3.1.4 Communications interfaces (Islam)

Not relevant, since no network is used, and all data is kept on the local computer.

3.2 System Features

3.2.1 Configure and Start a Game (Fahid, Rami)

3.2.1.1 Purpose

Produces a playable game from the selected configuration.

3.2.1.2 Stimulus/Response Sequence

The user selects New Game, and the system opens game setup. The user chooses the number of players and the level, enters each player's name and color, and clicks Start. If the input is valid, the system builds the board, chooses the first player and opens the game screen.

3.2.1.3 Functional Requirements

ID	Functional requirement	Imp.	Rationale
FR-09	The player shall choose the number of players (2, 3 or 4) and the level (Easy, Medium or Hard).	3	No game can start without these choices.
FR-10	Each player shall enter a name of 1 to 15 characters (letters, digits and spaces) that differs from the other players' names, ignoring letter case.	3	Names identify the players in the ranking, so they must be valid and unique.[TD]
FR-11	Each player shall choose a color no other player has chosen; the player's castle, pieces and home lane are shown in it.	3	Players must be able to tell their pieces apart.
FR-12	Start shall be enabled only when every player has a valid name and a color.	3	Invalid names or colors must not start a game.
FR-13	The system shall choose the first player at random, each player equally likely.	2	Any starting rule would work, so a random start is not essential.[TD]
FR-14	The board of each level shall have the tracks and special squares of table 1; each color has a 6-square home lane.	3	The customer's board specification.
FR-15	The four castles, start squares and arrow squares shall be in the fixed positions of the project description.	3	The customer's board specification.
FR-16	In every new game, the protection, question and surprise squares shall be placed at random, only on regular squares of the shared tracks, never on a start, arrow or home square or in a home lane.	3	The customer requires a new board in every game.

FR-17	No two special squares shall share a square, though they may be adjacent. Every board shall have exactly the number of special squares of table 1.	3	A board with missing or overlapping special squares would break the rules.
FR-18	Each special square type shall have its own icon and color, explained in the Rules window (FR-07).	3	Players cannot apply a square's rule without recognizing it.

Table 1 – Board tracks, special squares and dice of each level

Item	Easy	Medium	Hard
Square Numbers	52 squares	116 squares	188 squares
Protection squares	4	8	12
Arrow squares	4	8	12
Question squares	0	6 (2 easy, 2 medium, 2 hard)	9 (3 easy, 3 medium, 3 hard)
Surprise squares	0	2	3
Start squares	4	4	4
Dice	Step die	Step die and Boost die	Step die and UltraBoost die

3.2.2 Play a Ludo Game (Ghazal, Rami, Islam, Fahid)

3.2.2.1 Purpose

Covers gameplay from the first turn until completion.

3.2.2.2 Stimulus/Response Sequence

The player clicks Roll; the system shows the dice results and highlights the pieces that can move. The player selects one, and the system moves it and applies the rules of the squares on its way, including questions and surprise events on Medium and Hard. The turn then passes, unless the player rolls again; when all players but one have finished, the system shows the ranking and saves the game.

3.2.2.3 Functional Requirements

ID	Functional requirement	Imp.	Rationale
FR-19	After each turn, the turn shall pass clockwise to the next player, following the castle order on the board.	3	The game cannot continue without a fixed turn order.[TD]
FR-20	The player shall roll all dice of the level (table 1) together by clicking Roll.	3	Every move starts with a roll of the dice.

FR-21	Each die shall give a whole number in its range (step 1–6, Boost 1–3, UltraBoost 1–6), all values equally likely.	3	A biased die would make the game unfair.
FR-22	The number of steps shall be the step die result on Easy, and the step die result multiplied by (N − 1) on Medium and Hard, where N is the Boost or UltraBoost result. A result of 0 moves no piece.	3	Without this rule, the number of steps for a roll would be undefined.
FR-23	A piece in its home lane shall move by the step die result only, while a piece on a shared track moves by the full result.	3	Multiplied results would usually overshoot the finish, leaving home-lane pieces stuck.
FR-24	A 6 on the step die shall let the player move a piece from the castle to its start square, whatever the Boost or UltraBoost result. The player may instead use the full result to move a piece already on the board.	3	A basic rule of Ludo.
FR-25	A piece shall not leave the castle while an opponent's Double Block stands on its start square.	3	Without it, the Double Block rule (FR-35) would break.
FR-26	If any of the player's pieces can move by the result, the player shall move one of them, using the whole result.	3	Players may not skip a move or split a roll between pieces.
FR-27	A 6 on the step die shall give the player another roll after the move, even if no piece could move. Eating or bringing a piece to the finish shall not give another roll.	3	The extra roll after a 6 is a basic rule of Ludo.
FR-28	A third 6 in a row on the step die in the same turn shall be cancelled, whatever the Boost or UltraBoost result, and the turn shall pass to the next player.	3	A basic rule of Ludo.
FR-29	If no piece can move, the system shall show a 2-second message saying so, with the current player's name, and then pass the turn, unless a 6 gives another roll (FR-27).	3	The game must continue when a player cannot move; the 2 seconds are our choice.[TD]
FR-30	If the player does not roll or move for 30 seconds, the system should show a countdown; after 60 seconds the turn passes and any unused	1	Keeps the game moving if a player walks away, but

	roll is lost. The timer pauses while a question, surprise or Rules window is open.		the game works without it.[TD]
FR-31	After a roll, the system shall highlight, and allow selecting only the current player's pieces that can move by the result.	3	Lets players make only the moves that the rules permit.[TD]
FR-32	A piece shall move clockwise from its start square along the outer track. At an arrow square of its own color it turns at once, even before a full lap, onto the next inner track or into its home lane; for other colors an arrow square is a regular square.	3	A rule specified by the customer.
FR-33	A piece that stops on an unprotected square holding a single opponent piece shall eat it, sending it to its castle.	3	A basic rule of Ludo.
FR-34	Pieces on protected squares (protection, start, home, surprise) shall not be eaten. Pieces of different players may stand on the same protected square, unless one of them has a Double Block there (FR-35).	3	A basic rule of Ludo.
FR-35	Two or more pieces of one player on any square shall form a Double Block, which opponents shall not stop on or pass in either direction.	3	A basic rule of Ludo.
FR-36	If a roll would make a piece stop on or pass an opponent's Double Block, the player shall move another piece that can use the whole result. If no other piece can, the blocked piece shall stop just before the block.	3	A rule specified by the customer.
FR-37	A backward move shall stop at the piece's own start square.	3	Keeps a backward move on the piece's route.
FR-38	A home lane shall hold only its color's pieces and no special square but its home square.	3	A rule specified by the customer.
FR-39	A piece shall reach the finish only with the exact number of steps; a smaller result moves it closer, and a larger one does not move it.	3	A basic rule of Ludo.
FR-40	When a piece stops on a special square after any move, the system shall apply the square's effect at once, first sending a single opponent piece there back to its castle if the square is unprotected.	3	Without a fixed order, eating and the square's effect could conflict.

FR-41	When all four of a player's pieces reach the finish, the system shall record the player's place; the player is then skipped, and finished pieces affect no one.	3	The players' places are needed for the final ranking.
FR-42	The system shall show all players a random question of the square's level with four answers; only the player who stopped there answers, with no time limit.	3	Questions are how the game helps players practice the course material.
FR-43	A correct answer shall move the piece forward, and a wrong one back, by 1, 2 or 3 steps for an easy, medium or hard question.	3	Rewards correct answers, in line with the game's educational purpose.
FR-44	After an answer, the system shall show the result, the correct answer and an explanation until the player clicks Continue.	2	Helps players learn from mistakes, but the game works without it.[TD]
FR-45	A question shall not repeat in a game until all questions of its level are used; they then restart in a new random order.	2	Adds variety, but the game works without it.[TD]
FR-46	The system shall draw one event with the following probabilities: jump to the finish 10%, move an opponent 10%, back to the castle 10%, back 3 steps 30%, lose the next turn 30% and extra roll 10%.	3	Required by the customer; surprise events can't work without it.
FR-47	The system shall show the drawn event in a window and apply it when OK is clicked.	2	Makes each event visible to the players; the game works without it.[TD]
FR-48	When the event is "jump to the finish", the piece shall jump to the finish, ignoring Double Blocks and the exact-number rule.	3	One of the six surprise events set by the customer.
FR-49	When the event is "move an opponent", the player shall choose an opponent's piece on a shared track and roll the step die; that piece moves back half the result, rounded up. If that opponent has no movable piece, the player picks another; if none has one, the event is cancelled.	3	One of the six surprise events set by the customer.
FR-50	When the event is "back to the castle", the piece shall return to its castle.	3	One of the six surprise events set by the customer.

FR-51	When the event is "back 3 steps", the piece shall move back 3 squares, following the rules for eating, Double Blocks and the backward limit (FR-33, FR-35, FR-37).	3	One of the six surprise events set by the customer.
FR-52	When the event is "lose the next turn", the system shall show a message and skip the player's next turn. Any extra roll still due from a 6 shall also be lost.	3	One of the six surprise events set by the customer.
FR-53	When the event is "extra roll", the player shall at once roll the step die and the UltraBoost die and may move any piece, including leaving the castle on a 6. This roll is separate from the player's run of 6s: it does not count toward three 6s in a row, and a 6 on it gives another roll.	3	One of the six surprise events set by the customer.
FR-54	The game shall end when all players but one have finished; the remaining player takes last place. The duration runs from the click on Start to the end.	3	Defines when the game ends, which the final ranking and the saved record depend on.
FR-55	At the end, the system shall show the final ranking and the duration.	3	Required by the customer.
FR-56	A game left before it ends shall not be saved.	2	Keeps the history to finished games; the system works either way.

3.2.3 View Game History (Ghazal)

3.2.3.1 Purpose

Lets users view past games.

3.2.3.2 Stimulus/Response Sequence

The user selects Game History on the opening screen, and the system lists all saved games, which the user can sort and filter.

3.2.3.3 Functional Requirements

ID	Functional requirement	Imp.	Rationale
FR-57	The history screen shall list all saved games, newest first, with every history field (section 3.6).	3	Without this screen, saved games could not be viewed.
FR-58	The history screen should let the user sort the games by date, level or duration and filter them by level.	1	Makes a long history easier to browse, but it is not required.[TD]

3.2.4 Manage the Question Bank (Ghazal, Islam)

3.2.4.1 Purpose

Lets the admin maintain the question bank.

3.2.4.2 Stimulus/Response Sequence

The admin logs in and opens the question wizard. The system lists the questions; the admin adds, edits or deletes a question, and the system checks the change and saves it.

3.2.4.3 Functional Requirements

ID	Functional requirement	Imp.	Rationale
FR-59	The question bank that the admin manages shall be delivered with at least 30 questions on the course topics, at least 10 of each level.	3	The customer requires at least 30 questions; 10 per level is our choice.[TD]
FR-60	To open the question wizard, the admin shall log in with a username and a password.	3	Only the admin may view the question bank.
FR-61	The system shall have a single admin account (username and password), delivered with initial credentials that are given to the course staff with the submission.	3	The customer requires protected admin access; one account is enough.[TD]
FR-62	Inside the question wizard, the admin shall be able to change the password by entering the current password and the new one twice; the new password must meet NFR-31.	2	Lets the admin replace the initial password, but the wizard works without it.[TD]
FR-63	The wizard shall list all questions with their level and topic.	3	The admin needs the full list to choose a question to edit or delete.
FR-64	The wizard should let the admin search questions by text, filter by level and topic, and sort by level, topic or ID.	1	Eases managing a large bank; the customer did not explicitly ask for it.[TD]
FR-65	The admin shall add questions one at a time, each with a unique ID assigned by the system, a text (10–300 characters), four different answers (1–100 characters each), one correct answer, an explanation (1–300 characters), a level and a topic.	3	The customer requires adding questions; the explanation field supplies the explanation shown in FR-44.[TD]
FR-66	The admin shall be able to edit every field of a question, including its answers.	3	The customer requires editing questions.
FR-67	Deleting a question shall require confirmation and shall be refused if fewer than 30 questions in total, or fewer than 10 of its level, would remain.	3	Keeps enough questions for Medium and Hard games to run.[TD]

FR-68	Every change to the question bank shall be saved at once and apply from the next game.	3	Changes to the question bank must not be lost.

3.3 Performance Requirements (Islam, Fahid, Ghazal, Rami)

The time limits below apply on the weakest computer that can run Windows 11 [6]: a 64-bit, 2-core, 1 GHz processor, 4 GB of memory and a 720p screen, running Java 21; a game that meets them there meets them on any newer computer.

ID	Non-functional requirement
NFR-09	The opening screen shall appear within 5 seconds of program start. TD
NFR-10	The system shall show a visible reaction to every click within 0.1 seconds [8].
NFR-11	The game screen shall appear within 1 second of clicking Start, including building the board [8].
NFR-12	The dice results shall appear within 1 second of clicking Roll [8].
NFR-13	Each move shall be shown square by square and shall take at most 1 second in total, whatever its length [8].
NFR-14	Question and surprise windows shall open within 0.5 seconds after the piece stops [5].
NFR-15	The system shall keep every saved game and every question, with no limit on their number.
NFR-16	Saving a finished game shall take at most 1 second [8].
NFR-17	The history screen shall open within 1 second of selection [8], for a history of up to 1,000 games. TD

3.4 Design Constraints (Rami)

ID	Non-functional requirement
NFR-18	The system shall be written in Java 21.[5]
NFR-19	The screens shall be built with Java Swing.[6]
NFR-20	The code shall be modular and follow MVC: the Model, View and Controller classes shall be in separate packages, Model classes shall contain no screen code, and View classes no game rules.
NFR-21	The code shall use four design patterns, each documented with its classes and reason: Singleton for the single question bank, Factory for creating squares by type, Observer for updating the screens when the game changes, and Strategy for each level's dice rules. TD
NFR-22	The code shall follow the Java naming conventions.

[5] Java 21 was chosen because it is an LTS version.
[6] Java Swing is part of standard Java, so players install nothing extra.

NFR-23	Every version shall be delivered as a runnable JAR file, tested on at least two different computers before submission.

3.5 Software System Attributes (Fahid, Islam)

3.5.1 Reliability

ID	Non-functional requirement
NFR-24	If an unexpected error occurs, the system shall show an error message instead of closing, without losing saved games or questions.
NFR-25	The system shall reject invalid input with a message.
NFR-26	If the history file is missing, the system shall create a new one; damaged lines shall be skipped, and their number shown in a message. TD
NFR-27	If the question bank file cannot be read, the system shall offer only the Easy level and explain why in a message. TD
NFR-28	The dice and the surprise draws shall follow their stated probabilities: in 6,000 rolls of each die and 1,000 draws, a chi-square test at the 1% significance level shall find no deviation. TD
NFR-29	Each game rule shall have at least one unit test, including its edge cases, and all unit tests shall pass before each delivery.

3.5.2 Availability

Not relevant, since the system is an offline program with no server.

3.5.3 Security

ID	Non-functional requirement
NFR-30	An admin playing a game shall be treated like any other player and shall not see the answers or open the question wizard until the game ends. TD
NFR-31	The admin password shall have at least 15 characters [7] and shall be stored only as a salted PBKDF2 hash, which standard Java provides.
NFR-32	After a failed login, the system shall show only "Login failed: wrong username or password", not which one is wrong [9].
NFR-33	After each failed login in a row, the waiting time before the next attempt shall double, from 1 second up to 60 seconds. Only a successful login shall reset it; restarting the program shall not [7]. TD
NFR-34	The question bank file shall be encrypted with AES-GCM, which Java includes, so it cannot be read outside the game, and any change to it is detected (NFR-27).

3.5.4 Maintainability

ID	Non-functional requirement
NFR-35	All three levels shall run on the same code. The level settings and surprise probabilities shall be in one configuration class.
NFR-36	Each class shall have one clear responsibility.
NFR-37	Every public class and method shall have a Javadoc comment.

3.5.5 Portability

ID	Non-functional requirement
NFR-38	The same runnable file shall run without changes on Windows 11 and on macOS 13 or later, with Java 21 or later.
NFR-39	All data files shall be located relative to the program folder, with no fixed disk paths.

3.6 Other Requirements – Logical Database Requirements (Rami)

The system keeps two kinds of data: the questions and the admin account, which change only when the admin changes them, and the game history, which grows with every finished game. Each history record has these fields of the course template, in this order [2]:

- **gameID** – the game number: a whole number from 1
- **difficulty** – easy, medium or hard
- **timestamp(DD/MM/YYYY; HH:MM:SS)** – when the game ended
- **1st_place_name**, **2nd_place_name**, **3rd_place_name**, **4rd_place_name** – the players' names by finishing place, spelled as in the template; a place with no player is empty
- **game_duration(HH:MM:SS)** – the duration of the game, from start to end

ID	Functional requirement	Imp.	Rationale
FR-69	For each finished game, the system shall save one record with these fields, in this order.	3	Saved games must match the customer's provided template.